LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 05– RELASI ANTAR KELAS



Nama : Gendis Pambayun

NM: 103132400031

Kelas: PS1SD-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. **Concept Utama Diagram Kelas *(Class Diagram)***

Diagram kelas *(class diagram)* merupakan salah satu jenis diagram struktur statis dalam Unified Modeling Language (UML) yang merepresentasikan struktur, komponen, serta hubungan antar kelas dalam suatu sistem berorientasi objek

Di dalam diagram UML, sebuah kelas disimbolkan dalam bentuk kotak bertingkat yang terbagi menjadi tiga komponen utama:

a. Baris Pertama: Berisi nama dari kelas *(Class Name)*.
b. Baris Kedua: Berisi daftar atribut *(Attributes)* beserta tipe datanya.
c. Baris Ketiga: Berisi daftar operasi atau method *(Operations)* yang dimiliki oleh kelas tersebut.

Penentu Hak Akses *(Access Modifier)*

Tingkat aksesibilitas atribut maupun method dari luar kelas dinyatakan menggunakan simbol penentu hak akses (access modifier) sebagai berikut:

a. \+ *(Public)*: Elemen dapat diakses secara bebas dari dalam maupun luar kelas.
b. \- *(Private)*: Elemen hanya dapat diakses secara internal di dalam kelas tempat ia dideklarasikan.
c. # *(Protected)*: Elemen dapat diakses oleh kelas itu sendiri serta seluruh sub-kelas (turunannya).

2. **Hubungan Antar Kelas *(Class Relationships)***

Dalam Pemrograman Berorientasi Objek (Object-Oriented Programming / OOP), kelas-kelas saling berinteraksi dan terhubung satu sama lain. Terdapat empat jenis relasi utama antar kelas:

a. Asosiasi *(Association)*

Pengertian & Karakteristik: Hubungan struktural tingkat dasar antara dua kelas independen yang saling berkomunikasi melalui objek masing-masing. Hubungan ini bersifat longgar *(loosely coupled)*, di mana objek dari masing-masing kelas dapat diciptakan dan berdiri sendiri. Dalam kode program, kelas satu menggunakan method milik kelas lain (misalnya melalui penyerahan parameter pada method) tanpa menjadikan objek tersebut sebagai atribut permanen.

Contoh (Modul 5): Interaksi antara kelas Dosen dan kelas Mahasiswa, di mana kelas Dosen memiliki method giveScore(Mahasiswa s, int nilai) untuk memproses nilai tanpa menyimpan objek Mahasiswa sebagai atribut tetap.

Simbol UML: Digambarkan dengan garis lurus tanpa panah khusus yang menghubungkan kedua kelas.

b. Agregasi *(Aggregation)*

Pengertian & Karakteristik: Bentuk asosiasi khusus yang menggambarkan hubungan bagian dan keseluruhan (part-of / whole-part). Pada agregasi, kelas wadah *(whole)* menampung objek bagian sebagai atribut (misalnya menggunakan struktur data array). Meskipun terhubung, siklus hidup kedua objek bersifat independen; jika kelas utama/wadah dihapus, objek bagian di dalamnya tetap ada.

Contoh (Modul 5): Kelas Jurusan menampung kumpulan objek Mahasiswa dalam bentuk array Daftar. Objek Mahasiswa dibuat terpisah terlebih dahulu sebelum dimasukkan ke dalam objek Jurusan.

Simbol UML: Digambarkan dengan garis yang memiliki simbol belah ketupat kosong *(unfilled diamond)* pada sisi kelas wadah *(whole)*.

c. Komposisi *(Composition)*

Pengertian & Karakteristik: Bentuk agregasi yang memiliki keterikatan sangat kuat *(strongly coupled)*. Objek bagian merupakan komponen internal yang tidak dapat berdiri sendiri di luar objek utama *(whole)*. Siklus hidup objek bagian terikat penuh pada kelas utama; apabila objek utama dihancurkan/dihapus, maka seluruh objek bagian di dalamnya akan ikut musnah.

Contoh (Modul 5): Kelas Komputer yang tersusun atas atribut komponen CPU, Monitor, dan Mouse. Objek-objek komponen tersebut diinstansiasi secara langsung di dalam konstruktor kelas Komputer saat objek Komputer diciptakan.

Simbol UML: Digambarkan dengan garis yang memiliki simbol belah ketupat hitam/berisi *(filled diamond*) pada sisi kelas utama *(whole)*.

d. Generalisasi *(Generalization)*

Pengertian & Karakteristik: Hubungan hirarki antar kelas yang merepresentasikan konsep pewarisan *(inheritance)*. Hubungan ini menggambarkan hubungan is-a antara kelas induk *(superclass)* dan kelas turunan *(subclass)*.

Simbol UML: Digambarkan dengan garis panah dengan ujung segitiga kosong yang mengarah dari kelas turunan ke kelas induknya.

B. Guided

1. Guided 1

a. Mahasiswa

```
package com.gendispambayun.guided.guided1.asosiasi;


public class Mahasiswa {


    private String nim;
    private String nama;
    private int nilai;


    //setter
    public void setNama(String nama) {
        this.nama = nama;
    }


    public void setNim(String nim) {
        this.nim = nim;
    }


    public void setNilai(int nilai) {
        this.nilai = nilai;
    }


    //getter
    public int getNilai() {
        return this.nilai;
```

```
    }

    public String getNama() {
        return this.nama;
    }

    public String getNim() {
        return this.nim;
    }
}
```

b. Dosen

```
package com.gendispambayun.guided.guided1.asosiasi;

public class Dosen {

    private String kodeDosen;
    private String namaDosen;
//Setter

    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }
```

```
//Getter

    public String getKodeDosen() {

        //PEMROGRAMAN BERORIENTASI OBJEK 27
return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
        // ini asosiasi, method milik class Student dipanggil di class Dosen

// tp objek Student tidak menjadi atribut dr class Lecture
s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
        // ini asosiasi, method milik class Student dipanggil di class Lecture

// tp objek Student tidak menjadi atribut dr class Lecture
return s.getNilai();
    }
```

```
    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");
        Dosen d = new Dosen();
        d.giveScore(m, 90);
        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Output :

```
Nim :130118383
Nama :Budi
Nilai :90
------------------------------------
BUILD SUCCESS
```

2. Guided 2

    a. Jurusan

```
package com.gendispambayun.guided.guided2.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[41];

    //constructor
    public Jurusan (String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }
```

```
    //Behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " +  this.kodeJurusan);
        System.out.println("Nama Jurusan: " +  this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for(int i  = 0; i<jumlahMahasiswa;i++){
            System.out.println(i+1 + ". " +
                    daftarMahasiswa[i].getNIM()+" " +
                    daftarMahasiswa[i].getNama());
        }
    }
    //getter
    public String getKodeJurusan(){
        return kodeJurusan;
    }

    public String getNamaJurusan(){
        return namaJurusan;
    }
    //setter

    public void setKodeJurusan(String kodeJurusan){
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }
}
```

b. Mahasiswa

```
package com.gendispambayun.guided.guided2.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    //constructor
    public Mahasiswa(String NIM, String Nama){
```

```
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM(){
        return NIM;
    }
    public String getNama(){
        return Nama;
    }

    //Setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

c. Main (guided 2 dan 3)

```
package com.gendispambayun.modul05;

import com.gendispambayun.guided.guided2.agregasi.*;
import com.gendispambayun.guided.guided3.komposisi.*;

public class Main {

    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }

    public static void runGuided2() {
        Mahasiswa m1 = new Mahasiswa("1031324000331", "Gendis");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);
```

```
        prodi.displayMahasiswa();

    }

    public static void runGuided3() {
        Komputer pc = new Komputer("Intel Core i5", 3200, "LG 24 Inch",
"Logitech", "Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output :

```
Kode Jurusan: SD
Nama Jurusan: Sains Data
Daftar Mahasiswa:
1. 1031324000331 Gendis
2. 103112400149 John
```

3. Unguided 3

    a. Komputer

```
package com.gendispambayun.guided.guided3.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor,
String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }

}
```

b. Monitor

```
package com.gendispambayun.guided.guided3.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

c. Mouse

```
package com.gendispambayun.guided.guided3.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

d. Processor

```
package com.gendispambayun.guided.guided3.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }
```

```
    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

e. Main (guided 2 dan 3)

```
package com.gendispambayun.modul05;

import com.gendispambayun.guided.guided2.agregasi.*;
import com.gendispambayun.guided.guided3.komposisi.*;

public class Main {

    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }

    public static void runGuided2() {
        Mahasiswa m1 = new Mahasiswa("1031324000331", "Gendis");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();

    }

    public static void runGuided3() {
        Komputer pc = new Komputer("Intel Core i5", 3200, "LG 24 Inch",
"Logitech", "Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output :

```
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
-----------------------------------
```

C. Unguided

1. Soal Unguided

Soal — Sistem Manajemen Pesanan Cafe

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama *(Main)*.

Output yang Diharapkan

```
None
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

Jawaban :

a. Kasir

```
package com.gendispambayun.unguided.cafe;


public class Kasir {


    private String namaKasir;


    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
```

```
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] items = pesanan.getDaftarItem();
        int jumlah = pesanan.getJumlahItem();

        for (int i = 0; i < jumlah; i++) {
            totalSubtotal += items[i].getRincian().getHargaDasar();
            totalPajak += items[i].getRincian().getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + jumlah);
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("================================");
    }
}
```

b. ItemPesanan

```
package com.gendispambayun.unguided.cafe;

public class ItemPesanan {

    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return namaItem;
    }

    public RincianHarga getRincian() {
        return rincian;
    }
}
```

c. Pesanan

```
package com.gendispambayun.unguided.cafe;

public class Pesanan {
```

```
private int nomorMeja;
private ItemPesanan[] daftarItem; // Array penampung (Agregasi)
private int jumlahItem;

public Pesanan(int nomorMeja, int kapasitas) {
    this.nomorMeja = nomorMeja;
    this.daftarItem = new ItemPesanan[kapasitas];
    this.jumlahItem = 0;
}

public void tambahItem(ItemPesanan item) {
    if (jumlahItem < daftarItem.length) {
        daftarItem[jumlahItem] = item;
        jumlahItem++;
    }
}

public void tampilkanPesanan() {
    System.out.println("=== DAFTAR PESANAN CAFE ===");
    System.out.println("Nomor Meja: " + this.nomorMeja);
    for (int i = 0; i < jumlahItem; i++) {
        ItemPesanan item = daftarItem[i];
        System.out.println((i + 1) + ". " + item.getNamaItem() + " - Rp "
                + item.getRincian().getHargaTotal()
                + " (Harga: " + item.getRincian().getHargaDasar() + ", Pajak: "
                + item.getRincian().getPajak() + ")");
```

```
        }
        System.out.println();
    }

    public int getNomorMeja() {
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem;
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}
```

d. RincianHarga

```
package com.gendispambayun.unguided.cafe;

public class RincianHarga {

    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
```

```
        this.pajak = hargaDasar * 0.1; // Pajak 10%
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public double getPajak() {
        return pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

e. Main

```
package com.gendispambayun.unguided.cafe;

public class Main {

    public static void main(String[] args) {
// 1. Membuat 1 objek Kasir
        Kasir kasir = new Kasir("Budi");
// 2. Membuat 3 objek ItemPesanan secara independen (Agregasi)
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
```

```
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);
// 3. Buat 1 objek Pesanan untuk Meja 4 (kapasitas 5 item)
        Pesanan pesananMeja4 = new Pesanan(4, 5);
// 4. Masukkan ketiga objek ItemPesanan ke dalam objek Pesanan
        pesananMeja4.tambahItem(item1);
        pesananMeja4.tambahItem(item2);
        pesananMeja4.tambahItem(item3);
// 5. Menampilkan rincian pesanan
        pesananMeja4.tampilkanPesanan();


        kasir.prosesPembayaran(pesananMeja4);
    }
}
```

Output :

```
=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
================================
```

Penjelasan :

Berdasarkan soal unguided sistem manajemen pesanan kafe pada Modul 5, perancangan kode sumber *(source code)* menerapkan tiga konsep relasi antar kelas utama dalam PBO, yaitu komposisi, agregasi, dan asosiasi.

Relasi komposisi diterapkan antara kelas ItemPesanan dan RincianHarga, di mana objek rincian harga diinstansiasi secara langsung di dalam *constructor* item pesanan sehingga siklus hidupnya terikat mutlak. Sementara itu, hubungan agregasi dibangun

pada kelas Pesanan yang menampung sekumpulan objek ItemPesanan independen melalui sebuah array dan method tambahItem().

Selanjutnya, relasi asosiasi diimplementasikan pada kelas Kasir yang berinteraksi dengan kelas Pesanan melalui parameter method prosesPembayaran(Pesanan pesanan) tanpa menyimpannya sebagai atribut permanen. Seluruh alur program ini dirangkai di dalam kelas Main sebagai titik awal eksekusi, mulai dari pembuatan objek kasir oleh Budi, instansiasi menu pesanan meja 4, hingga pencetakan rincian pesanan dan struk pembayaran akhir secara terstruktur.

D. Kesimpulan

Praktikum Modul 5 ini memberikan pemahaman dalam membangun relasi antar kelas yang terdiri dari komposisi, agregasi, dan asosiasi. Melalui latihan pembuatan sistem manajemen pesanan kafe, praktikan dapat mengimplementasikan hubungan antar kelas secara nyata di dalam bahasa pemrograman Java, mulai dari pengelolaan komponen objek yang terikat secara mutlak hingga interaksi antar objek independen.

Secara keseluruhan, praktikum ini berhasil melatih keterampilan logika pemrograman dalam merancang struktur class diagram dan mengaplikasikannya ke dalam bentuk *source code* yang terenkapsulasi dengan baik. Setiap jenis relasi seperti kelas ItemPesanan yang menerapkan komposisi dengan RincianHarga, kelas Pesanan yang menerapkan agregasi, serta kelas Kasir yang menggunakan asosiasi saling mendukung untuk menciptakan program yang terstruktur, rapi, dan sesuai dengan skenario operasional di dunia nyata.